

Game Licensing Architecture

Vigno Smart Class · app.dat, Sessions & the TTL — how download-lane games are protected

1. The protection layers (at a glance)

A download-lane game is protected by five independent locks:

- Encrypted at rest — the downloaded file (.enc) is AES-256-GCM encrypted; useless without the key.
- Decryption key never on disk — fetched from the server per play, held in memory only (online-only).
- Device-locked game — an in-game check (LicenseGuard) quits if the build runs on a different machine.
- Session-enforced — login is required to play; logging out kills a running game.
- Traceable — a moving email watermark ties any screen capture back to the buyer.

2. Where app.dat comes from

- At Play, the launcher calls the server: POST /content/:id/game-license.
- The server SIGNS a token (RSA private key in backend/keys/) bound to this PC's machine GUID + a short expiry.
- The launcher WRITES that token into the game's _Data folder under a disguised name (app.dat).
- The game's LicenseGuard READS it at startup and verifies signature + machine + expiry — quits if any fail.

In short: signed on the server, written to disk by the launcher, verified inside the game. app.dat is NOT the decryption key — it only proves device authorization.

3. Why there is a time dependency (the TTL)

Because the game verifies app.dat OFFLINE. LicenseGuard runs inside the game and only does local checks (signature, machine, expiry) — it never calls the server.

To know "is the session still logged in?", the game would have to ASK the server. But only the launcher talks to the server; the game does not. The only freshness signal an offline check can verify is TIME — compare the token's expiry to the clock.

That is the entire reason a TTL exists: time is the only "is this still valid?" check a game can do without phoning home. The token TTL is 2 hours (configurable via GAME_LICENSE_TTL_HOURS), and the launcher mints a fresh token on every play — so legit launches always have a valid one, while a captured copy dies within ~2 hours.

4. How login / logout is enforced

The session IS wired in — at the launcher level, which is where the server connection lives:

Session event	Enforced?	How
Must be logged in to start a game	Yes	Launcher needs a live session to fetch the key + token; no login != no decrypt/launch.
Logout while a game is running	Yes	Launcher kills the running game and wipes its temp folder + app.dat immediately.
A captured app.dat reused later, offline	Partly	Covered by the 2-hour TTL (the one thing the launcher can't reach).

5. The one gap the TTL covers

Everything the launcher controls is tied to the session. The only residual risk is a

COPY of app.dat that someone extracted while a game was running. The game checks that copy offline, so the only way to kill it is to let it expire — hence the TTL.

And that copy is weak on its own: app.dat is not the decryption key and is bound to one machine. To abuse it, an attacker would also need the decrypted game files (capturable only during play), the same machine, and to act within 2 hours.

6. Optional: true session-binding (dropping the TTL)

There is exactly one way to make logout invalidate instantly with no TTL: have the GAME itself call the server at startup to validate the session live, instead of reading a local file.

Trade-offs to weigh before choosing this:

- The game then needs its OWN internet connection on every launch (not just the launcher).
- More integration work inside the game (auth, server URL, error handling, offline UX).
- A determined cracker can still patch out that network call — same limit as the offline check.
- If the server is down, nobody can play.

This is why the standard design (and what is built here) keeps an offline check + a short TTL: games stay fast, launchable, and not dependent on their own server link.

7. Current security posture (summary)

- Encrypted at rest; decryption key never written to disk (online-only, in memory).
- Device-locked game binary (LicenseGuard) — a copied build won't run elsewhere.
- app.dat regenerated every play, expires in 2 hours, wiped on logout and on game exit.
- Decrypted files exist only during play, then reliably deleted (with retry + sweeps).
- Every copy watermarked / traceable to the buyer.

Honest limit (true of ALL PC DRM, Steam and Denuvo included): a determined

reverse-engineer can strip the in-game check from the binary. The only way to fully close that is cloud-streaming the game (files never reach the user). Everything short of that is deterrence + traceability — which this design maximizes.

